

A Systems Software Stack for Enabling AI Model Inference on the Atalla AI Accelerator

Robert Zhang ECE 37920 zhan4808@purdue.edu	Mary Francis ECE 37920 franci92@purdue.edu
Soumil Verma ECE 37920 verma178@purdue.edu	Jiayi Liu Volunteer liu3988@purdue.edu

Spring 2026

The feedback for the report is to be returned to:

Robert Zhang
zhan4808@purdue.edu

This project was mentored by the following Teaching Assistants:

Akshath Ravikiran, araviki@university.edu
Sooraj Chetput, schetput@purdue.edu

A Systems Software Stack for Enabling AI Model Inference on the Atalla AI Accelerator

Robert Zhang
ECE 37920

zhan4808@purdue.edu

Mary Francis
ECE 37920

franci92@purdue.edu

Soumil Verma
ECE 37920

verma178@purdue.edu

Jiayi Liu
Volunteer
liu3988@purdue.edu

Abstract

SoCET’s AI accelerator, Atalla, is built with specialized hardware to accelerate machine learning (ML) workloads. However, hardware alone is insufficient for achieving end-to-end model execution on Atalla. Rather, model inference requires a systems software layer that efficiently maps high-level neural network operations to low-level implementations, called kernels. With no existing abstraction for Atalla’s tile-centric instruction set architecture (ISA) and programming model, ML models would require direct ISA-level programming, severely limiting usability and scalability. We present a kernel library enabling AlexNet and ViT models to execute on Atalla without low-level C++ or assembly programming. We map matrix operations (general matrix multiplication (GEMM)) and non-linear operations (ReLU, softmax) to Atalla’s ISA. Matrix kernels keep weights resident in the systolic array while streaming input activations through it, reducing redundant weight loads. Software pipelining overlaps data transfers with computation to hide memory latency and improve throughput. Convolution kernels accumulate partial sums across tiles using on-chip buffers to minimize scratchpad writes and memory traffic. Element-wise kernels exploit vector-lane parallelism to maximize computational concurrency and reduce latency. We evaluate our library in the context of model inference, comparing latency, accuracy, and throughput of models running with our kernel library on Atalla against equivalent PyTorch implementations on our functional simulator. Because we lack the physical chip, evaluation is limited to a functional simulator. At its core, our kernel library establishes the foundations for programmable and scalable ML inference on Atalla. Future extensions will explore operator fusion and expanded kernel coverage to further improve utilization and performance.

1 Introduction and Background

Atalla Ax01 is a domain-specific accelerator for machine-learning inference built around a Systolic Array, Vector Core, Scalar Core, and SDMA engine executing a VLIW instruction stream. While the hardware targets efficient tiled linear algebra, hardware alone does not provide a usable inference platform. Without supporting software, running even a single model layer requires writing Atalla ISA programs directly: scheduling DMA transfers, invoking compute units, managing

scratchpad placement, and constructing legal VLIW packets.

This does not scale to modern models. A transformer forward pass contains many matrix multiplications, normalizations, reductions, and attention operations [1], each with different dataflow and tiling requirements. Manually implementing every operator for every shape would make model deployment slow, error-prone, and difficult to maintain.

At the start of this project, three software gaps separated Atalla hardware from end-to-end inference. First, the chip was still in fabrication, so no physical target existed for software development or verification. Second, no reusable kernel library existed for common ML operators such as GEMM, convolution, softmax, or normalization. Third, no model-level compilation path existed to lower PyTorch models into executable Atalla programs.

This report presents the Systems Software stack developed to close those gaps. Our main contributions are:

1. A functional simulator for Atalla ISA programs with software models of the major execution units and per-kernel metrics collection.
2. A shared scheduling and packetization flow used by both handwritten assembly kernels and compiler-generated AtallaC code.
3. A kernel library covering nine common operators, including GEMM, convolution, softmax, attention, layer normalization, and pooling.
4. Kernel-level optimizations including software pipelining and loop unrolling for improved packet efficiency and memory overlap.
5. An AtallaC kernel path compiled through AtallaCC and compared directly against handwritten kernels.
6. A `torch.fx`-based graph compiler that lowers PyTorch graphs to tiled Atalla programs, demonstrated end-to-end on AlexNet [2] and ViT-Micro [3].

This work builds on parallel efforts from the hardware teams, which developed the execution units and memory system, and from the compiler team, which developed AtallaCC, a PPCI-based compiler from AtallaC to Atalla assembly. Prior systems-software efforts provided an early functional simulator and basic kernel infrastructure. Our work extends that

foundation with broader operator coverage, optimized kernels, cross-validation between handwritten and compiled code paths, and a model-level deployment flow.

The remainder of the report first summarizes the architectural constraints that shape kernel software (section 2), then describes the simulator, kernel implementations, compiler flow, and experimental results.

2 Architectural Overview

This section summarizes the hardware constraints that determine how the Atalla software stack must tile, schedule, and verify kernels.

2.1 Architecture and Memory Hierarchy

Atalla Ax01 contains four software-visible execution units: a Systolic Array, Vector Core, Scalar Core, and SDMA Engine. The Systolic Array executes tiled BF16 matrix multiply-accumulate operations from scratchpad and accumulates partial products in an on-chip PSUM buffer. When a GEMM or convolution kernel uses this path correctly, partial sums stay in the PSUM buffer across the K loop and the output tile is written to scratchpad only once.

The Vector Core handles element-wise BF16 operations, reductions, masking, and approximate exponential operations used by softmax and GELU [?]. The Scalar Core handles loop control, address arithmetic, and branches. The SDMA Engine moves tiles between DDR and the 2 MB scratchpad asynchronously, allowing memory transfers to overlap with compute when the instruction stream is software-pipelined.

Scratchpad capacity determines legal tile sizes. For a GEMM with BF16 elements ($B_e = 2$ bytes), two input tiles and one output tile must fit:

$$B_e(T_M T_K + T_K T_N + T_M T_N) + B_{\text{staging}} \leq S_{\text{scpad}}. \quad (1)$$

If the output tile is held in the PSUM buffer, the $T_M T_N$ term is removed. This is the key reason PSUM-aware kernels reduce scratchpad traffic relative to AtallaC kernels that spill accumulators.

2.2 VLIW ISA Constraints

Atalla programs execute as VLIW packets with four slots: Systolic, SDMA, Vector, and Scalar. Each packet can issue at most one operation per slot, so two independent vector operations still cannot execute in the same packet. Empty slots are filled with NOPs.

The hardware also provides no dynamic hazard handling: no forwarding, out-of-order issue, or dependency checking. The packetizer must separate dependent operations by the producer latency; otherwise the program may execute incorrectly without a hardware-visible fault. These constraints make packetization a software responsibility rather than a backend detail.

Table 1: ISA constraints most relevant to kernel scheduling.

Operation class	Slot	Example	Latency
Matrix MAC	Systolic	<code>gemm.vv</code>	40
Weight load	Systolic	<code>lw.vi</code>	0
Vector load/store	Vector	<code>vreg.ld/st</code>	15/2
Vector arithmetic	Vector	<code>add.vv, mul.vv</code>	4–5
Vector reduction	Vector	<code>rsum, rmax</code>	20
Scalar branch	Scalar	<code>bne, blt</code>	1
DDR transfer	SDMA	<code>scpad.ld/st</code>	520

2.3 Design Requirements for Efficient Kernels

These architectural constraints lead to four kernel requirements.

Use the PSUM buffer. For tiled GEMM and convolution, spilling partial sums after each K -tile adds avoidable scratchpad traffic. PSUM-aware kernels write the output tile once, while spill-based kernels write after each accumulation step.

Overlap SDMA with compute. Because DDR transfers are much longer than vector or systolic operations, kernels should prefetch tile $k+1$ while computing tile k . In the functional simulator this does not directly reduce measured cycle time, but it exposes the schedule needed for hardware execution.

Reduce scalar loop overhead. Every tile-loop iteration consumes scalar instructions for induction updates and branches. Loop unrolling reduces this overhead and exposes more independent operations to the packetizer.

Expose legal co-issue. The packetizer can only fill multiple slots when the instruction stream contains independent operations from different slot classes. Kernel structure therefore determines whether VLIW parallelism is available.

2.4 Target Models and Kernel Library Motivation

We evaluate the kernel library on two scaled inference graphs chosen for operator coverage rather than production accuracy. AlexNetSmall preserves the convolution-heavy structure of AlexNet [2] using 32×32 inputs and reduced channel widths. ViT-Micro is a single-block Vision Transformer with `dim=32`, `n_tokens=32`, and a $2 \times$ FFN expansion, exercising GEMM, LayerNorm, Softmax, GELU, attention, and residual adds characteristic of Transformer inference [3]. Together, the two models cover the main CNN and Transformer operator classes while remaining small enough for functional simulation.

fig. 1a and fig. 1b show the model graphs and Atalla kernel mappings. Shaded nodes dispatch to the functional simulator; unshaded nodes either require no kernel, such as reshapes and transposes, or currently fall back to NumPy references.

Table 2: Kernel coverage across the two evaluation models.

Kernel	AlexNet	ViT	Current path
Tiled GEMM	✓	✓	simulator
Convolution	✓		simulator
ReLU	✓		simulator
Max Pooling	✓		simulator
Element-wise Add	✓	✓	simulator
Softmax		✓	simulator
LayerNorm		✓	simulator
GELU		✓	NumPy ref
Fused SDPA		✓	simulator

3 Software Stack Implementation

3.1 Functional Simulator

3.1.1 Simulator Design and Execution Model

Because the Atalla chip was still in fabrication, kernel development needed a software target for correctness testing before RTL or silicon were available. A cycle-accurate RTL simulator would eventually be needed for timing validation, but it was incomplete when this work began and is too slow for rapid kernel iteration. We therefore used a functional simulator as the primary development target: it executes Atalla instruction semantics against software models of architectural state and completes kernel tests in seconds rather than minutes.

The simulator models the state visible to Atalla programs: DDR, scratchpad, scalar registers, vector registers, mask registers, and the Systolic Array PSUM buffer. On each step, the simulator fetches one VLIW packet, decodes its slots, and dispatches each operation to the corresponding unit model. The Systolic Array model performs scratchpad-resident BF16 matrix multiply-accumulate into the PSUM buffer; the SDMA model copies bytes between DDR and scratchpad; the Vector model executes element-wise, mask, reduction, and exponential operations; and the Scalar model handles address arithmetic, loop control, and branches. fig. 2 shows this execution model.

3.1.2 Static Image Format and Binary Compatibility

The simulator executes a kernel as a static image containing both instructions and data. `build_compiler` combines a packetized `.in` instruction stream with a data section generated by `dram_writer.py`. The data section places inputs, weights, and output buffers at fixed DDR offsets, so the simulator can initialize DDR before execution and read the output tensor after the kernel terminates.

The same `.in` image format is used by the RTL simulator and hardware scheduler flow. This binary compatibility matters because kernels debugged in the functional simulator can later be submitted to RTL without changing the program representation; only the execution model changes from functional semantics to cycle-level timing.

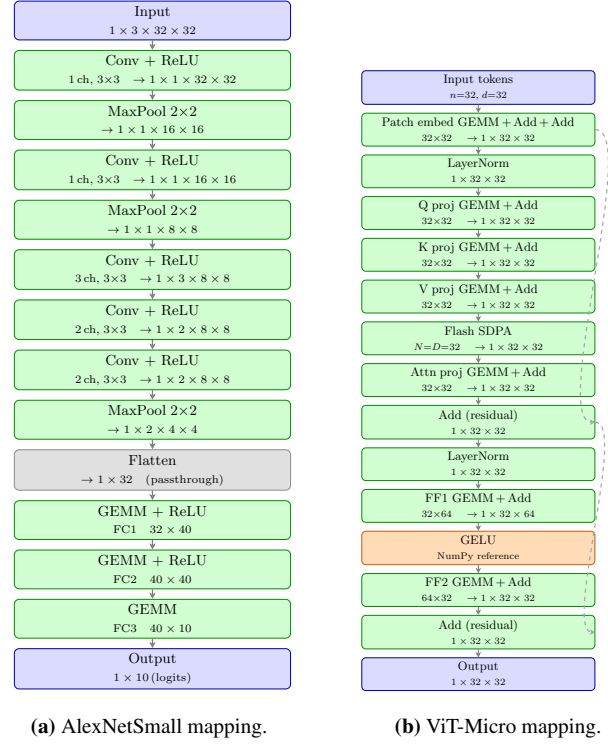


Figure 1: AlexNetSmall graph and Atalla kernel mapping. Convolution layers use implicit im2col; fully connected layers map to tiled GEMM. ViT-Micro graph and Atalla kernel mapping. GELU and fused attention currently fall back to NumPy references pending hardware kernel integration.

3.1.3 Emission, Scheduling, and Packetization

There are three stages in the simulator flow: *kernel emission*, *packetization*, and *simulation*. Kernel emission produces an instruction stream from either handwritten assembly or AtallaC compiled through AtallaCC. Packetization groups those operations into legal VLIW packets. Simulation executes the resulting packet stream.

Handwritten assembly kernels are reported in two modes. In *structural emission*, the kernel emits a conservative packet stream, often with one useful operation per packet. This mode is useful for correctness and memory-traffic measurements because the schedule is explicit. In *DAG-packed emission*, the handwritten instruction stream is passed through a greedy dependence-based packer that places independent operations into the same packet when they use different slot classes and satisfy latency constraints.

Compiled AtallaC uses a separate front end: AtallaCC lowers AtallaC source to flat Atalla assembly, and `build_compiler` invokes `compile_asm()` to produce the packetized `.in` image. `compile_asm()` tracks register dependencies, inserts producer latency constraints, and greedily fills packets subject to the one-SDMA, one-Systolic, one-Vector, and one-Scalar structural limit.

For the matched C-vs-handwritten comparison in section 3.3, both paths use the same `compile_asm()` packetization flow. Thus, differences in packet count and slot utilization

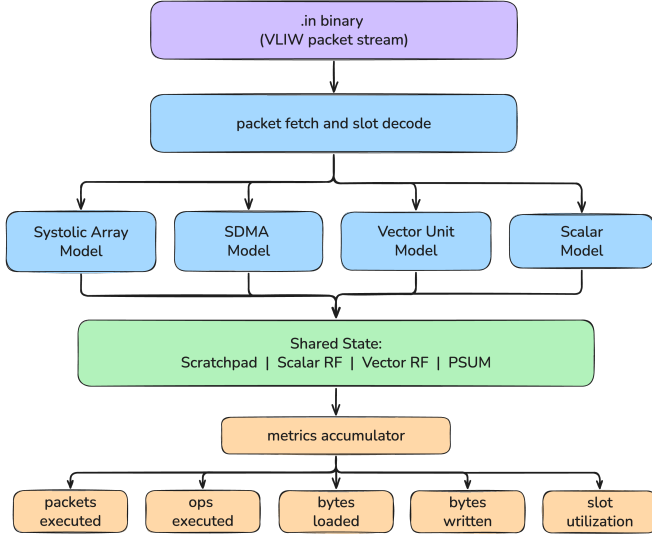


Figure 2: Functional simulator internal structure. Each VLIW packet is decoded into per-slot operations dispatched to execution unit models sharing a scratchpad and register state. The metrics accumulator records statistics across the full kernel run.

reflect the emitted instruction streams, not different packetizers. Earlier handwritten results labeled “structural” or “DAG-packed” should be read as handwritten-assembly scheduling experiments, not AtallaCC results.

3.1.4 Metrics

The simulator collects structural execution metrics for each kernel run. These metrics are useful for comparing schedules and memory traffic, but they are not cycle-accurate timing measurements.

Table 3: Functional simulator metrics.

Metric	Meaning
Packets executed	Dynamic VLIW packets retired (issue proxy, not cycles)
Ops executed	Non-nop operations across all slots
Slot utilization	ops/(4 × packets), four-slot fill
Bytes loaded / written	SDMA loads / stores
FLOPs	Model work: tile MACs (systolic), vector FU work, 1 op per scalar op
Arithmetic intensity	FLOPs/(bytes loaded + bytes written) where bytes count only memory traffic from SDMA loads and stores

These metrics must be interpreted with the simulator’s limits in mind. Packet count is not runtime because the functional simulator does not model stalls, contention, or exact unit latency. Slot utilization measures static scheduling density, but a fuller packet is not necessarily faster if it creates a latency hazard in hardware. Arithmetic intensity captures DDR-scratchpad traffic only; it does not include scratchpad-register movement. Cycle-accurate RTL remains necessary to

convert these structural metrics into timing results.

3.2 Handwritten Assembly Kernels

3.2.1 Kernel Structure

The handwritten assembly library was developed directly against the Atalla ISA using the functional simulator. Each kernel explicitly manages SDMA transfers, scratchpad placement, vector masks, systolic array invocations, PSUM accumulation, and output writeback. This path is lower-level than AtallaC, but it exposes hardware features that the current compiler cannot yet use automatically, especially explicit mask control and PSUM-buffer accumulation.

3.2.2 Kernel Implementations

The handwritten assembly library covers vector element-wise kernels, reduction kernels, systolic kernels, and attention kernels. The main implementation challenge is not the mathematical definition of each operator, but mapping that definition onto Atalla’s available ISA features: vector masks, scalar fallbacks, the Systolic Array, and the PSUM buffer.

Element-wise kernels: ReLU and sigmoid. ReLU is implemented with vector masks:

$$f(x) = \max(0, x).$$

For an input vector X , the kernel first creates a mask whose bit i is set when $x_i > 0$. It then performs a masked vector add from X into a zero-initialized output vector. Active lanes copy the positive values; inactive lanes remain zero.

Sigmoid requires more work because Atalla has vector exponentiation but no vector division:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

The kernel negates X with `mul.vs`, applies `exp.vi` to compute e^{-x_i} , and adds one. To compute the reciprocal, each element is moved through the Scalar Core, where scalar reciprocal is available, then written back into the correct vector lane using a one-hot mask. This implementation is functional but exposes an ISA limitation: element-wise reciprocal is expensive without vector division or vector reciprocal support.

Reduction kernels: softmax, LayerNorm, and max pooling. Softmax first computes a row maximum for numerical stability, subtracts it, exponentiates with `exp.vi`, reduces the row sum, and normalizes. The online softmax variant [4] keeps running maximum and normalization statistics, avoiding one full input pass. In our structural metrics, online softmax has much higher AI than standard softmax because the recurrence performs more arithmetic over the same measured DDR traffic; this should be read as a compute-to-byte difference, not as a direct latency improvement.

LayerNorm performs three row-wise passes: mean, variance, and normalization. The sequential version issues these passes in order. The latency-aware version interleaves independent work across rows, for example issuing a vector load

for row $k + 1$ while a reduction for row k is in flight. This adds scheduling bubbles in the functional simulator, but it represents the kind of latency-hiding schedule needed on hardware.

Max pooling iterates over pooling windows and reduces each window with vector element-wise max operations. Unlike purely sequential structural kernels, max pooling naturally exposes some independent operations, so its structural emission can exceed the 25% one-op-per- packet baseline.

Systolic kernels: tiled GEMM and convolution. The tiled GEMM kernel decomposes $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$ into $T \times T$ tiles and iterates over the K dimension. Weight tiles are loaded into the Systolic Array, activation tiles stream through, and partial products accumulate in the PSUM buffer across all K/T iterations. Scratchpad is written only once per output tile. fig. 4 illustrates this weight-stationary dataflow.

The convolution kernel uses a channel-first implicit im2col mapping [5]. Each filter is treated as a weight matrix and each input patch as an activation column, allowing the inner product to run through the Systolic Array. Partial sums accumulate in the PSUM buffer across input channels before one scratchpad write per output location.

Attention kernels. The standard attention kernel computes $\text{softmax}(QK^T/\sqrt{d})V$ using tiled matrix products and an in-line softmax normalization step. The FlashAttention kernel [6] instead applies the tiled online softmax recurrence across sequence tiles, maintaining running max and sum statistics and applying correction factors at tile boundaries. This avoids materializing the full $N \times N$ score matrix in scratchpad.

3.2.3 Kernel Verification

Each kernel is verified by a reproducible three-stage flow. First, bounded random inputs are generated with a fixed seed and written into the static image data section. Second, the corresponding packetized .in program is executed by the functional simulator, producing DDR, scratchpad, scalar-register, vector-register, and mask-register outputs. Third, the DDR output tensor is compared against a PyTorch or NumPy BF16 reference for the same input. Correctness is measured using elementwise error and cosine similarity, with tolerance set by expected BF16 rounding.

3.2.4 Software Pipelining, Tiling, and Loop Unrolling

The handwritten kernels also serve as a place to test optimizations that the current AtallaC compiler does not yet perform.

Software pipelining overlaps memory movement with compute. A non-pipelined tiled GEMM issues the SDMA load for tile k , waits for the data, and then computes tile k . The pipelined version issues the prefetch for tile $k+1$ while the Systolic Array computes tile k , so the memory transfer can be hidden behind useful work on hardware. This requires a prologue to issue the first prefetch and an epilogue to drain the final iteration.

Double buffering enables this overlap at two levels. Scratchpad double buffering alternates between scratchpad regions so one tile can be consumed while the next tile is

loaded. Register double buffering applies the same idea between scratchpad and vector registers or systolic inputs, so the next row or tile can be staged while the current one is used.

Instruction interleaving is used for kernels without a simple load-compute-store loop, such as LayerNorm. Independent work from different rows is interleaved; for example, a vector load for row $k+1$ can be issued while a reduction for row k is in flight.

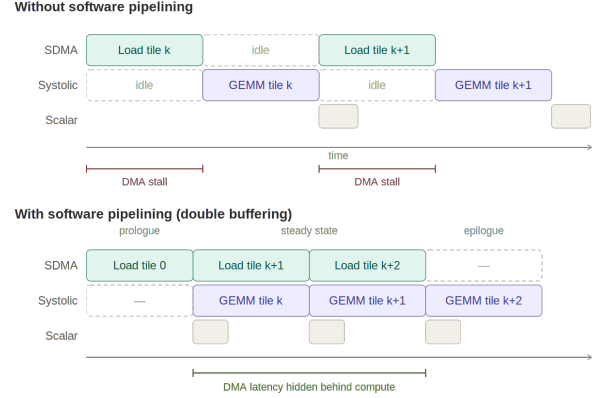


Figure 3: Software pipelining overlaps the load for the next tile with computation on the current tile. The functional simulator captures the instruction schedule but not the cycle-level latency benefit.

Tiling breaks large tensors into scratchpad-resident blocks. For GEMM, the handwritten assembly uses a weight-stationary schedule: weights are loaded into the Systolic Array and reused while activation tiles stream through. Partial sums accumulate in the PSUM buffer, avoiding scratchpad round trips until the final output tile is ready.

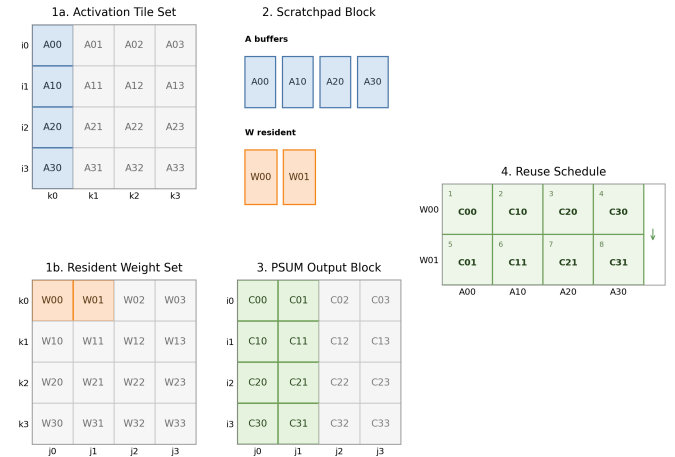


Figure 4: Weight-stationary GEMM tiling. Weight tiles are reused across activation tiles, while partial sums accumulate in the PSUM buffer before a single output writeback.

Loop unrolling reduces scalar branch overhead and exposes more independent operations to the packetizer. Instead of executing a branch and induction update every iteration, an unrolled loop performs several tile steps per branch group. In

Table 4: Handwritten assembly kernel metrics in structural emission mode. *Instrs* is the static instruction count; *Dyn Pkts* is the dynamic packet count. Structural emission is intentionally conservative, so slot utilization is omitted.

Kernel	Instrs	Dyn Pkts	AI	Bytes Ld	Bytes Wr
LayerNorm (seq)	354	354	1.80	2,048	2,048
Softmax	396	396	1.27	2,048	2,048
Softmax (online)	22,192	22,192	102.50	2,048	2,048
ReLU	239	239	0.77	2,048	2,048
Sigmoid	37	37	2.09	64	64
GEMM (tiled)	115	220	32.69	384	128
Conv (tiled)	111	262	16.98	464	32
Attention	108	10,524	2.06	69,632	2,048
FlashAttention	2,643	2,579	7.89	12,288	6,144
Add	26	243	0.17	4,096	2,048
MaxPool	160	282	21.54	128	32

handwritten assembly this also gives the packetizer a larger window of independent operations from different slot classes. AtallaCC does not currently perform this transformation, so unrolling was applied manually in the handwritten assembly kernels.

3.2.5 Results: Handwritten Assembly Kernels

table 4 reports handwritten kernel metrics in *structural emission* mode. In this mode, most kernels emit a conservative packet stream with one useful operation per packet; slot utilization is therefore near the 25% single-slot baseline except where the handwritten schedule explicitly places independent operations in the same packet. These results therefore emphasize arithmetic intensity, memory traffic, and dynamic packet count rather than scheduler quality.

The structural results show two useful trends. First, PSUM-aware systolic kernels have high arithmetic intensity because partial sums are accumulated on chip and written once. Second, online softmax has much higher arithmetic intensity than standard softmax in this microbenchmark because the recurrence performs more arithmetic while moving the same number of bytes. This does not imply lower latency in the functional simulator; it only indicates a different compute-to-byte ratio.

Table 5: Convolution slot utilization and packet count under structural and DAG-packed emission. DAG packing exposes legal co-issue across independent operations.

Variant	Structural		DAG-packed	
	Util	Pkts	Util	Pkts
Conv (base)	25.0%	343	29.2%	297
Conv + pipeline	25.0%	389	29.2%	357
Conv + pipe + unroll	25.0%	381	29.0%	341

To evaluate scheduling density, we also run selected handwritten kernels through *DAG-packed emission*. This is a handwritten assembly-side greedy packing experiment: independent operations are placed into the same VLIW packet when

they use different slot classes and satisfy dependence constraints. It should be interpreted as a packing experiment on handwritten instruction streams, not as an AtallaCC result.

DAG packing improves convolution slot utilization by about four percentage points and reduces packet count by 10–13%. Pipelining alone increases packet count in structural mode because the prologue and epilogue add real instructions; its benefit is expected to appear as latency hiding on cycle-accurate hardware, not as fewer functional-simulator packets. Combining pipelining with unrolling gives the best DAG-packed packet count by reducing scalar loop overhead while still exposing independent operations for co-issue.

3.3 AtallaCC Integration

3.3.1 Compiled AtallaC Path

AtallaCC [7] is the compiler team’s PPCI-based compiler [8] from AtallaC to Atalla assembly. In this work, AtallaC provides a higher-level path for writing kernels: the programmer expresses scratchpad buffers, tiling loops, and calls to Atalla-specific operations, while AtallaCC handles lowering, instruction selection, register allocation, and assembly emission.

The emitted `.s` file is then packetized by `compile_asm()` before execution on the functional simulator. For the comparison in this section, handwritten assembly and compiled AtallaC kernels both pass through the same `compile_asm()` packing flow. This matters because differences in packet count and slot utilization then come from the instruction streams themselves, not from different packetizers.

3.3.2 Compiled C vs. Handwritten Assembly

The AtallaC suite implements the same nine operator classes as the handwritten assembly library. All kernels match the NumPy BF16 reference at $\cos \geq 0.999$ except handwritten LayerNorm ($\cos = 0.993$) and AtallaC FlashAttention ($\cos = 0.977$), both discussed in section 4. The comparison below focuses on scheduling density and memory traffic. In all figures, “Generator” refers to the handwritten assembly path.

The central tradeoff is that AtallaC emits regular instruction streams that pack well under `compile_asm()`, while handwritten assembly can exploit hardware features the compiler does not yet expose. The most important case is GEMM: the handwritten path keeps partial sums in the Systolic Array PSUM buffer across the entire K -loop and writes each output tile once; AtallaC spills intermediate accumulations through scratchpad, increasing memory traffic even when its packets are denser.

fig. 5 shows that neither path dominates across all kernels. The handwritten advantage concentrates on operators whose correctness depends on careful interleaving of reductions, scalar updates, and loads. The AtallaC advantage concentrates on operators with regular, loop-dominated instruction sequences.

For GEMM packet count, AtallaC has the initial advantage: its base kernels execute 11–13% fewer packets because the

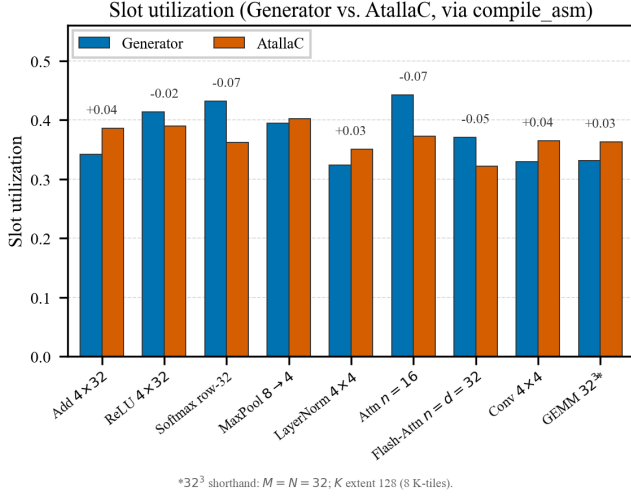


Figure 5: Slot utilization after matched `compile_asm()` packetization (both paths). Handwritten assembly leads on reduction-heavy kernels (softmax, attention, FlashAttention) where manual instruction ordering exposes independent reductions and loads for co-issue. AtallaC leads on regular arithmetic kernels where the compiler-emitted loop body presents more independent operations to the scheduler.

Table 6: GEMM arithmetic intensity (FLOPs/byte): handwritten assembly (Gen) vs. AtallaC (C), both via `compile_asm()`. The 1.38–1.51 $\times$ gap reflects PSUM buffer accumulation in the handwritten assembly vs. scratchpad spill in the AtallaC path. * $M=N=K=32$; ** $K=256$.

Size	Gen AI	C AI	Ratio
32 ^{3*}	13.0	8.6	1.51 $\times$
64 ^{3**}	14.5	9.8	1.48 $\times$
512 ³	15.3	11.1	1.38 $\times$
1024 ³	15.8	11.2	1.41 $\times$

regular tile body packs more densely. Handwritten pipelining alone increases packet count. The prologue and epilogue add real instructions but its intended benefit is hardware latency hiding, not fewer functional-simulator packets. Combined with 4 $\times$ loop unrolling, the handwritten path reduces packet count by 15%, recovering and slightly exceeding the AtallaC baseline by eliminating scalar branch overhead and widening the scheduler’s view of independent operations.

On memory efficiency, the handwritten path has the advantage. fig. 7 and table 6 show that handwritten assembly writes 4–32 $\times$ fewer bytes and achieves 1.38–1.51 $\times$ higher arithmetic intensity across the four GEMM sizes. The gap grows with matrix size: a 1024³ GEMM accumulates 32 K -tile iterations per output tile, each incurring an extra scratchpad write in the AtallaC path.

Taken together, the two advantages belong to different paths. AtallaC is more scheduling-efficient (denser packets at base level), while handwritten assembly is more memory-efficient (PSUM-aware accumulation). With pipelining and unrolling applied, the handwritten path eliminates the AtallaC packet-count advantage while retaining the memory ef-

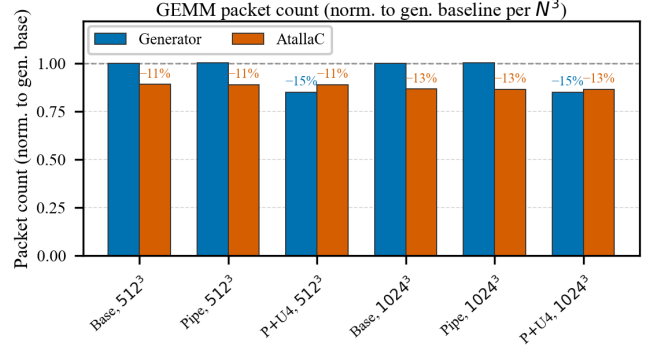


Figure 6: GEMM packet count normalized to the handwritten assembly baseline (both paths via `compile_asm()`). AtallaC base kernels execute 11–13% fewer packets at large sizes. Handwritten pipelining alone adds packets from prologue/epilogue; combined with 4 $\times$ unrolling it achieves –15%, removing the AtallaC scheduling advantage.

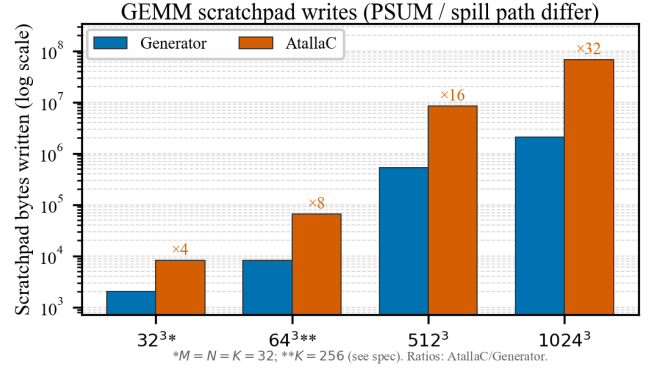


Figure 7: GEMM scratchpad bytes written (log scale). Handwritten assembly writes 4–32 $\times$ fewer bytes by accumulating partial sums in the PSUM buffer; AtallaC spills intermediate partial sums to scratchpad after each K -tile iteration. The ratio grows with matrix size because larger GEMMs require more K -tile iterations, each adding an extra write in the AtallaC path.

iciency advantage, making the fully optimized handwritten path strictly better on both metrics for large GEMM. For arithmetic-regular small kernels, however, AtallaC’s scheduling density advantage is meaningful and represents the expected behavior of a compiler that lacks PSUM-path awareness.

Closing the gap requires two additions to the AtallaC compilation path: an intrinsic or optimization pass that preserves GEMM partial sums across K -tile iterations without scratchpad round-trips, and compiler-level software pipelining and loop unrolling to match the packet-count reductions demonstrated in the handwritten kernels.

3.4 Atalla Graph Compiler

3.4.1 FX Graph Capture and Operator Matching

The graph compiler connects the standalone kernel library to model execution: instead of invoking kernels manually, it maps PyTorch graph nodes to AtallaC kernel templates and emits the corresponding tiled program. The graph compiler captures a PyTorch model’s computation graph using `torch.fx.symbolic_trace` [9], producing a flat operator-level graph. Each node is matched against a registry of AtallaC kernel stubs: `torch.mm` and `torch.linear` map to the tiled GEMM kernel; `functional.relu`, `softmax`, `layer_norm`, `max_pool2d`, and `element-wise add` map to their respective kernels.

Attention is handled specially. Rather than decomposing the attention sublayer into four primitive nodes (QK^T , scale, softmax, weighted sum), the model registers `AtallaSdpa` as a leaf module in the FX tracer, preventing symbolic decomposition. The resulting `atalla_sdpa` node provides a single dispatch point at which the validated flash attention kernel can be substituted, and we use that for emulation of `atalla_sdpa` on the functional simulator.

3.4.2 Tile Planning and Scratchpad Allocation

For each matched operator, tile dimensions are selected such that required input and output tiles co-reside within the scratchpad capacity constraint of eq. (1). Tile planning and scratchpad address allocation were implemented by Vihaan. Data movement between consecutive layers is handled through a static scratchpad address plan: each layer’s output is written to a fixed scratchpad region, and the following layer’s input SDMA loads from the corresponding DDR address.

3.4.3 AtallaC Code Generation

For each matched operator, the compiler emits an AtallaC kernel function with resolved tile dimensions as compile-time parameters, wrapping intrinsic calls in the tiling loop structure. The emitted source is compiled through `AtallaCC`, packetized by `compile_asm()`, and executed on the functional simulator, following the same path as the standalone AtallaC kernels in section 3.3. fig. 8 shows the full pipeline.

3.4.4 Results: AlexNetSmall and ViT-Micro

Both models are evaluated in chained validation mode, where functional simulator outputs feed forward as inputs to subsequent layers.

AlexNetSmall (`scale=1`, 32×32 input, 10 classes) dispatches 21 of 42 graph nodes to the functional simulator with zero NumPy fallback and 4 passthrough nodes (transposes, flatten). The final output achieves $\cos = 0.9999$ against the BF16 PyTorch reference, and the run passes strict end-to-end validation. Aggregate AI is 33.1 FLOPs/byte, reflecting the spatial data reuse of convolutional layers. Profiling figures for AlexNetSmall are omitted; the model is convolution-

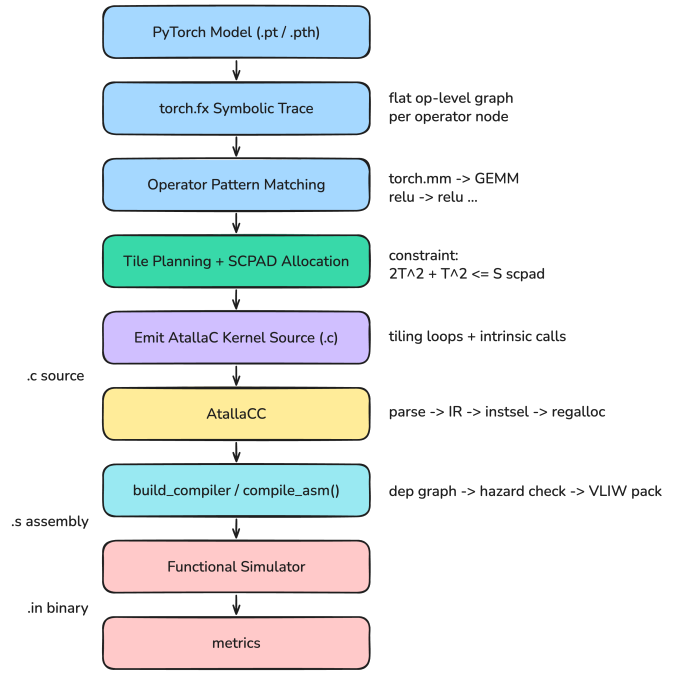


Figure 8: Atalla graph compiler end-to-end pipeline.

dominated at this scale and the per-layer breakdown provides limited additional insight beyond the aggregate metrics.

ViT-Micro (`dim=32`, `n_tokens=32`, single encoder block) dispatches 20 of 48 graph nodes to the functional simulator; GELU is the sole NumPy fallback. The Flash attention kernel (`ATALLA_SDPA_FLASH=1`) is fully emulated, decomposing the $O(N^2D)$ attention computation into tiled matrix-multiply and in-tile softmax packets. In chained validation the final output achieves $\cos = 0.978$; in oracle mode, where reference activations are reloaded before each operator, each emulated operator matches its reference output. The remaining chained-mode error comes from a known multi-tile element-wise add issue in the FFN path, discussed in section 4.

fig. 9 shows that Flash SDPA dominates DMA traffic, loading 133 kB versus at most 12 kB for any other emulated node. This comes from the FlashAttention tiled K/V re-read pattern: the score matrix is not materialized, but K and V tiles are re-visited across output blocks.

fig. 10 shows that Flash SDPA also dominates dynamic packet count, accounting for 54.1% of packets. LayerNorm contributes 17.3%, while all Add nodes together contribute only 10.6%, confirming that residual adds are not the scheduling bottleneck.

fig. 11 explains why aggregate slot efficiency alone is insufficient. GEMM templates are systolic-dominated, Add and LayerNorm are vector-heavy, and fused SDPA mixes systolic, vector, scalar, and memory operations due to its tiled QK-softmax-AV structure.

The nearly identical slot efficiency across both models suggests that the one-unit-per-slot VLIW structure and operator slot mix, rather than only graph type, bound aggregate utilization. The ViT profiling figures show that similar aggregate

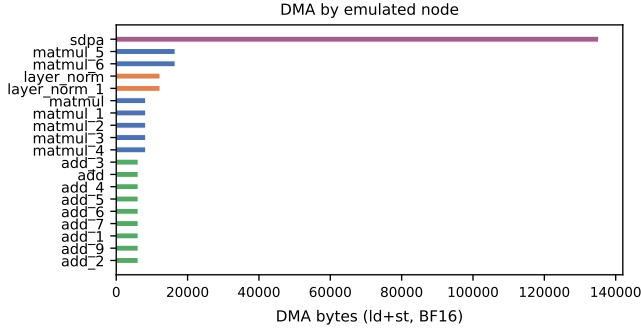


Figure 9: DMA bytes (load + store, BF16) per emulated node, ViT-Micro with Flash attention ($N=D=32$). The fused SDPA node accounts for 133 kB driven by the Flash tiled K/V re-read pattern; all other nodes transfer ≤ 12 kB.

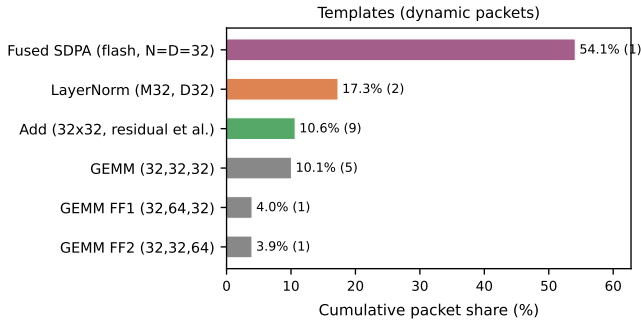


Figure 10: Cumulative dynamic packet share by operator template, ViT-Micro with Flash attention. Fused SDPA accounts for 54.1% of all packets; LayerNorm follows at 17.3%. Parenthesized counts are the number of graph nodes dispatched to each template.

utilization can arise from very different operator mixes, which is why per-template breakdowns are more informative than the end-to-end average alone.

4 Discussion and Limitations

4.1 Discussion

Across both handwritten assembly in the functional simulator and the AtallaC kernels, we continually notice that more independent instructions lead to better slot utilization. Loop unrolling creates more instructions that can run in parallel, which helps the packetizer fill more slots and potentially reduce packet count. We can see this in the handwritten assembly kernels, where software pipelining and loop unrolling reduces packet count by up to 15% for GEMM.

We notice that software pipelining affects optimization differently when done within the functional simulator vs. the compiler. In handwritten assembly, it helps overlap loads and compute, so that computations aren't bottlenecked by the hardware waiting for memory-accesses to finish. However, the functional simulator has no effect on reducing packet count, and instead could increase it due to prologue and epilogue in-

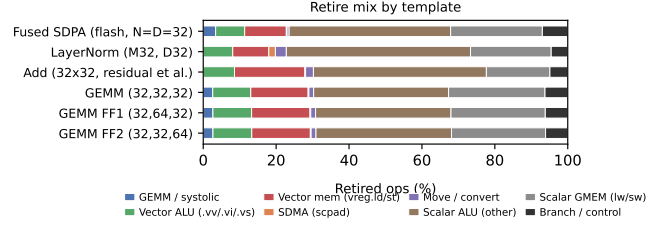


Figure 11: Retired operation mix by template, ViT-Micro. GEMM templates are systolic-dominated; Fused SDPA has a broader mix of systolic, vector ALU, and scalar ops from its tiled QK-softmax-AV structure; LayerNorm and Add are vector-ALU-heavy.

Table 7: End-to-end inference metrics on the Atalla functional simulator. AI = arithmetic intensity (FLOPs/byte).

Model	Nodes	Sim	NumPy ref	Cos	AI	Slot eff
AlexNetSmall	42	21	0	0.9999	33.1	0.278
ViT-Micro	48	20	1	0.978	3.6	0.279

structions being added. When writing kernels in AtallaC, software pipelining showed no measurable impact on the packet count or slot utilizations, which indicate these optimizations are not performed within the compiler.

Another notable difference is that the functional simulator utilizes the PSUM buffer for accumulation, reducing scratchpad writes that consequently increase arithmetic intensity. AtallaC kernels instead store intermediate results to the scratchpad, which reduces arithmetic intensity by increasing memory traffic. AtallaC kernels can still achieve higher slot utilization and fewer packets than the functional simulator, because the compiler can make it easier for the packetizer to schedule and pack instructions.

Despite the differences between the two interfaces for writing kernels, what remains consistent is that the structure of the kernel plays a large role in which approach performs better. Softmax, Attention, and Flash Attention benefit more from handwritten assembly, as the reduction operations can be placed with independent loads and operations to fill more slots by the packetizer. Kernels with heavy arithmetic, such as GEMMs and convolution, perform better in AtallaC due to emitting more independent operations. Therefore, while it can be more convenient to write kernels in C, it is not necessary that either one interface is better; it depends heavily on how well the kernel exposes parallelism naturally or requires manual ordering.

4.2 Limitations

A significant blocker for our work was the absence of a completed scheduler and packetizer in the compiler, along with the fact that PPCI does not support loop unrolling. This prevented the compiler from adequately using our optimizations to improve packet count and slot utilization. We got around this by using a shared `compile_asm()` scheduler and packetizer within the functional simulator. Although this is not integrated

within the compiler, it allows us to evaluate optimized kernel code and estimate their impact on packet count and slot utilization.

Additionally, the effectiveness of software pipelining is limited by scratchpad capacity and the partitions within it. The tile size and number of rows that can fit in each scratchpad restricts how much double buffering and overlap between memory transfers and computation can be achieved.

Our metrics also don’t capture the full picture of kernel computation. Arithmetic intensity only takes into account memory traffic between DRAM and scratchpad, but not between scratchpad and registers. Packetization also assumes every instruction costs the same but doesn’t accurately take into account the latency of SDMA.

Ultimately, the kernels we write and the models we run are limited by the size of the chip and the scratchpad capacity. All tiles must fit within the scratchpad, which restricts the maximum model size that can be run on the chip.

5 Conclusion and Future Work

We have presented a software stack enabling ML inference evaluation on the Atalla Ax01 AI accelerator before physical silicon is available. The stack spans five components: a functional simulator with execution unit models for the Systolic Array, SDMA engine, Vector Core, and Scalar Core; a shared VLIW scheduling and packetization tool used by both kernel paths; a nine-operator generator kernel library verified against NumPy reference outputs; an AtallaC kernel suite compiled through AtallaCC and cross-validated against the generators; and a `torch.fx`-based graph compiler that lowers PyTorch operator graphs to tiled AtallaC programs.

Handwritten kernels achieve 38–51% higher arithmetic intensity than AtallaC kernels for GEMM due to PSUM buffer utilization, while AtallaC kernels achieve 5% higher slot utilization and 11–13% fewer packets for large GEMM due to denser instruction emission. Combined software pipelining and $4\times$ loop unrolling reduces generator GEMM packet count by 15.0% at 512×512 and 1024×1024 . Both AlexNetSmall and ViT-Micro execute end-to-end through the graph compiler pipeline. Slot utilization is structurally bounded at 0.277–0.292 across all kernel types and both models, reflecting the one-unit-per-slot hardware constraint rather than scheduler quality.

The primary next steps are:

- To extend the kernel library further with BatchNorm, GELU, etc., and validate them functionally.
- Improve scheduling by incorporating software pipelining and loop unrolling for the AtallaC kernels.
- Implement efficient per-kernel tiling and tile reuse strategies during the Atalla graph-level memory planning phase.
- Run end-to-end models on Atalla, such as EfficientFormerv2 and AlexNet.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16×16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [4] M. Milakov and N. Gimelshein, “Online normalizer calculation for softmax,” *arXiv preprint arXiv:1805.02867*, 2018.
- [5] K. Chellapilla, S. Puri, and P. Simard, “High performance convolutional neural networks for document processing,” in *Tenth International Workshop on Frontiers in Handwriting Recognition*. Suvisoft, 2006.
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [7] M. Golovenchits, S. Patel, J. Erwin, S. Sarlashkar, and A. Hegde, “Atallacc: Designing a compiler stack for a programmable atalla ai accelerator,” SoCET Lab, Purdue University, Tech. Rep., 2026.
- [8] W. Bouwman, “PPCI: Pure Python compiler infrastructure,” <https://ppci.readthedocs.io>, 2023.
- [9] J. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel, “torch.fx: Practical program capture and transformation for deep learning in python,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2022.

A Teamwork

Work was divided between teammates mainly by interest. A summary of contributions by each team member can be found in table 8. For the report, each team member was responsible for writing up their part of the work on the project itself – for all major sections of the report. Additional contributions on the report from each member are also listed.

B Code Repository

The Functional Simulator source code is available at: https://github.com/Purdue-SoCET/atalla/tree/functional_sim/functional_sim

Team Member	Contribution to Project
Jiayi Liu	Functional Simulator ISA Updates, LayerNorm Kernel, Kernel Benchmarking
Soumil Verma	ReLU, Sigmoid, Naive Attention Kernels, Kernel Benchmarking
Mary Francis	GEMMs, Tiled GEMMs Kernels, Attention Layer, Kernel Benchmarking
Robert Zhang	Convolution, Flash Attention kernels, Atalla Graph to AtallaCC integration, Running AlexNet and ViT

Table 8: Team Member Contribution Summary

The Atalla Graph source code is available at: <https://github.com/Purdue-SoCET/atalla-models/tree/vihaanc-dev>

The full end to end model to functional simulator pipeline source code is available at: <https://github.com/Purdue-SoCET/atalla-models>